

Propuesta estructurada

Seminario Aplicado de Ciberseguridad - FDSI / SPTI

Hito 1 · 05 de septiembre de 2026

Campo	Información
Asignatura	FDSI
Código de equipo	FDSI-GP-02
Tema asignado	Aislamiento de herramientas utilizadas por agentes
Integrantes	[Completar nombres de los integrantes]
Repositorio / carpeta de trabajo	Pendiente de creación.

1. Problema de seguridad

Un agente puede usar herramientas para leer archivos, conectarse a un servicio o ejecutar una tarea. Si esas herramientas tienen más permisos de los necesarios, una instrucción equivocada o una herramienta mal utilizada podría acceder a información privada, cambiar archivos o consumir demasiados recursos. El riesgo no depende solamente de lo que responda el agente, sino también de lo que sus herramientas puedan hacer [1].

Queremos proteger los archivos del laboratorio y la disponibilidad del entorno de ejecución. Propondremos un agente simulado en Python que envíe llamadas definidas de antemano. Así podremos repetir las mismas entradas y concentrarnos en el aislamiento, sin depender de las respuestas variables de un LLM. Solo usaremos datos ficticios y un servicio HTTP propio; no realizaremos pruebas sobre terceros.

2. Pregunta de investigación

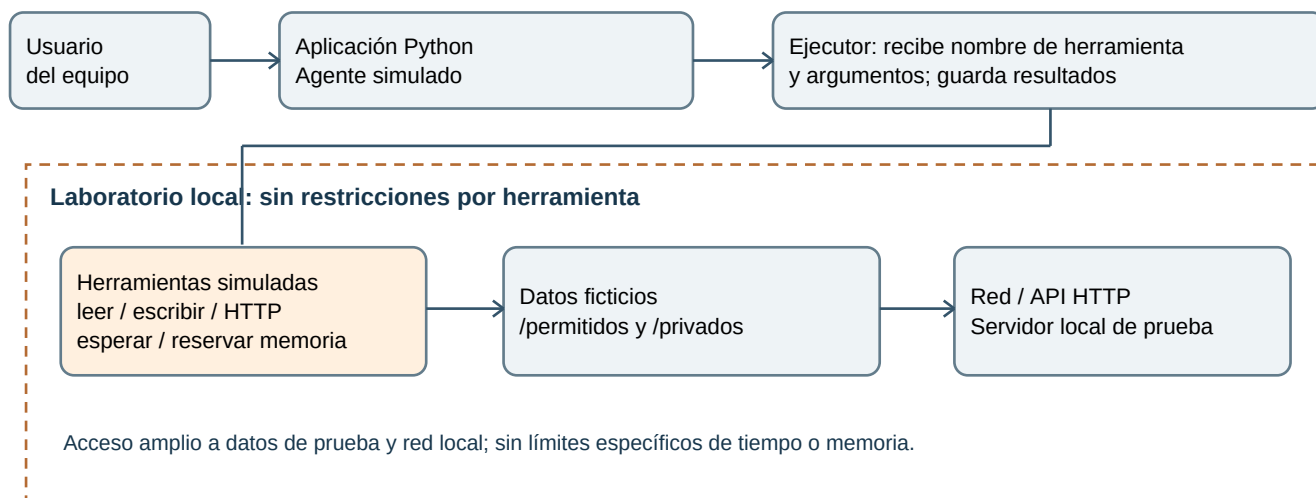
¿Qué cambia en el acceso a archivos, la comunicación por red y el consumo de tiempo y memoria cuando ejecutamos las mismas herramientas de un agente simulado con y sin restricciones de aislamiento?

3. Hipótesis

Esperamos que la versión Secure impida leer archivos privados, modificar archivos de entrada y contactar el servidor de prueba. También esperamos que detenga las tareas que superen el tiempo o la memoria permitidos, manteniendo la lectura de archivos autorizados. La versión Unsecure permitiría esas acciones dentro del laboratorio. Estos son resultados esperados, todavía no comprobados.

4. Arquitectura Unsecure

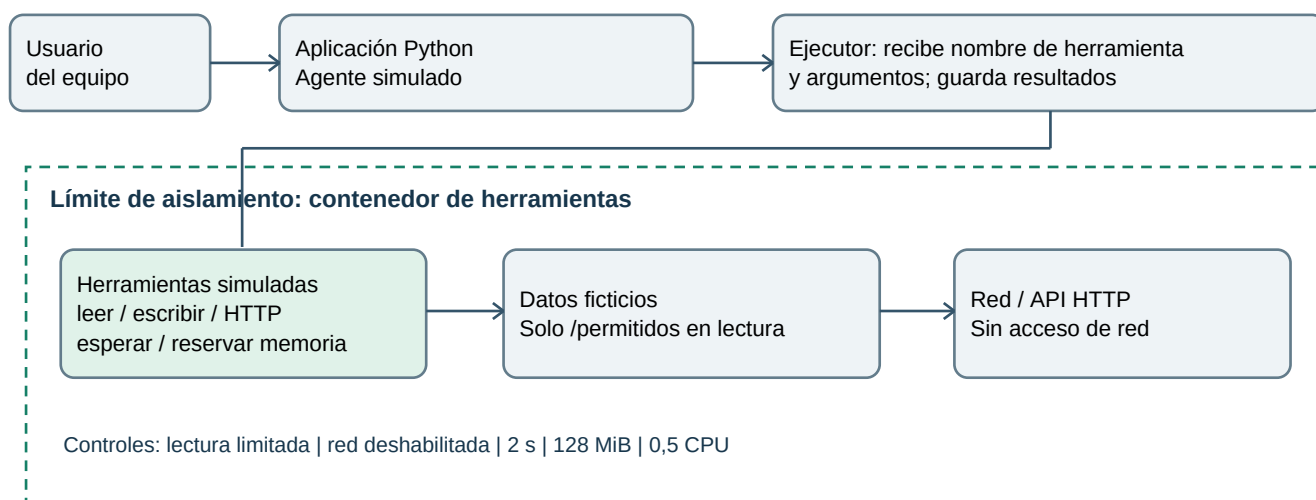
El usuario inicia una prueba en la aplicación. El agente simulado selecciona una herramienta y entrega sus argumentos al ejecutor. Las herramientas tendrán acceso de lectura y escritura a las carpetas ficticias y podrán contactar un servidor HTTP del laboratorio. No tendrán límites específicos por herramienta de tiempo, CPU o memoria.



El laboratorio completo será desechable y tendrá un límite general de 1 GiB de memoria y 1 CPU, además de una parada de emergencia de 15 segundos por prueba. Esta protección externa será igual para ambas versiones; no representa el control Secure que se quiere evaluar.

5. Arquitectura Secure

Mantendremos el agente, las herramientas, los archivos de entrada y las pruebas. Cambiará la ejecución: cada llamada se realizará en un contenedor desechable sin privilegios. Solo se montará la carpeta autorizada en modo lectura; la carpeta privada quedará fuera. El sistema de archivos del contenedor será de solo lectura y su red estará deshabilitada [2, 4].



El ejecutor terminará y eliminará el contenedor si la herramienta supera 2 segundos, medidos desde el inicio de la tarea. Se configurarán 128 MiB de memoria, sin memoria de intercambio adicional, y 0,5 CPU [3]. El agente no podrá cambiar estos controles. Los registros se guardarán desde el ejecutor, fuera del contenedor. El límite discontinuo del diagrama separa la herramienta del resto del laboratorio; el color verde identifica los controles añadidos.

6. Pruebas propuestas

Ejecutaremos cada prueba cinco veces por arquitectura (50 ejecuciones en total), en el mismo equipo y con la misma versión del código. Antes de cada ejecución se restaurarán los archivos y se limpiarán los registros del servidor. Los resultados de la tabla son previsiones.

ID / prueba	Condición / entrada	Esperado Unsecure	Esperado Secure
T01 · Lectura	Leer primero /permitidos/nota.txt y luego /privados/secreto.txt, ambos ficticios.	Lee ambos archivos.	Lee el autorizado; no obtiene el contenido privado.
T02 · Escritura	Intentar reemplazar el contenido de /permitidos/nota.txt.	Modifica el archivo.	Escritura rechazada; archivo intacto.
T03 · Red	Enviar un HTTP GET al servidor propio del laboratorio, con un ID de prueba.	El servidor recibe la solicitud y responde.	El servidor no recibe la solicitud.
T04 · Tiempo	Ejecutar una herramienta que espera 10 segundos.	Finaliza después de unos 10 s.	El ejecutor la detiene al superar 2 s.
T05 · Memoria	Reservar y escribir 256 MiB de memoria; mantenerlos durante 1 s.	Completa la reserva dentro del límite general.	No completa la reserva; se registra el límite de memoria o la terminación.

Registraremos ID, arquitectura, repetición, entrada, salida, error, duración y estado final del contenedor. Para T02 compararemos el hash SHA-256 del archivo antes y después; para T03 revisaremos el registro HTTP; para T05 revisaremos la configuración y el estado de memoria del contenedor, incluido OOMKilled cuando corresponda. Un error de código o de configuración no contará como bloqueo exitoso.

7. Criterios de evaluación

Métrica / criterio	Cómo se calcula / observa	Umbral esperado en Secure
Acceso correcto a archivos	T01: contar lecturas autorizadas exitosas y lecturas privadas exitosas.	5/5 lecturas autorizadas y 0/5 lecturas privadas.
Integridad del archivo	T02: comparar hashes antes y después en cada ejecución.	Archivo sin cambios en 5/5 casos.
Bloqueo de red	T03: contar solicitudes con ID recibidas por el servidor.	0 de 5 solicitudes recibidas.
Límite de tiempo	T04: medir desde el inicio de la herramienta hasta confirmar su detención.	5/5 detenidas en máximo 3 s: límite de 2 s + 1 s de tolerancia.
Límite de memoria	T05: comprobar límite de 128 MiB y contar reservas de 256 MiB completadas.	0/5 reservas completadas por restricción de memoria.

8. Tecnologías previstas

Python 3: aplicación, agente simulado y herramientas. Usaremos pathlib para archivos, urllib.request para HTTP, time para medir duración y hashlib para verificar cambios en archivos.

Docker Engine o Docker Desktop con contenedores Linux: entorno de ejecución y configuración de permisos, red, CPU y memoria. Usaremos subprocess desde el ejecutor para iniciar, detener e inspeccionar los contenedores. El contenedor de herramientas no tendrá acceso al socket de Docker.

Servidor HTTP local en Python: destino propio para comprobar el acceso de red. **JSON o CSV:** registros y comparación de resultados. **Git:** guardar código, configuraciones e instrucciones para repetir las pruebas. No se prevé usar un modelo LLM ni una API de pago en esta primera versión del experimento.

9. Referencias iniciales

Las siguientes fuentes orientarán la selección de controles y la configuración del experimento:

[1] OWASP. "LLM06:2025: Excessive Agency".

<https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>

[2] Docker. "Running containers".

<https://docs.docker.com/engine/containers/run/>

[3] Docker. "Resource constraints".

https://docs.docker.com/engine/containers/resource_constraints/

[4] Docker. "None network driver".

<https://docs.docker.com/engine/network/drivers/none/>

10. Declaración de uso de IA

Se utilizó ChatGPT (Codex) para revisar los lineamientos y la plantilla, organizar la propuesta, redactar el borrador, plantear los diagramas, las pruebas y las métricas, y localizar documentación técnica inicial. La IA también se utilizó para generar este PDF. No se han ejecutado pruebas ni se presentan resultados experimentales. Los integrantes deberán revisar el contenido y confirmar que comprenden el alcance antes de entregarlo.